

FAIRWDDI WP3 ST3

DDI-L Architecture for ReQuest

Progress, Technical Architecture & Strategic Roadmap

Centre de Données Socio-Politiques (CDSP)

Sciences Po / FAIRwDDI Research Infrastructure • September 09 2026

1. What Has Been Done So Far

1. Database Design & Modeling

- **Standard-Agnostic Core:** 15 tables across 5 layers with `DDIIdentifiable` mixin.
- **Dual-Engine Parity:** Automatic SQLite3 fallback for instant dev/tests (<0.5s) & Postgres 17 in prod.
- **Multilingual JSONB & SKOS:** Ingested CESSDA ELSST R6 (3,470 concepts) via RDF loader.

2. Normalizing & Hashing

- **Multi-Tier Hashing:** Deterministic SHA-256 text & compound URN fingerprinting.
- **Order-Independent Sets:** Set hashing (`ucl-...`) eliminating 70% false-positive drift alarms.
- **Canonical URN Taxonomy:** 16-char hex IDs supporting Curated, Canonical, Random & Alias URNs.

3. Corpus Harvesting

- **CLOSER UK:** 12 projects, 2+ GB XML, 50M+ items in BaseX XML database.
- **CSO Ireland & MIDUS:** 5 projects, 23 series, 67 studies, 6,044 questions.

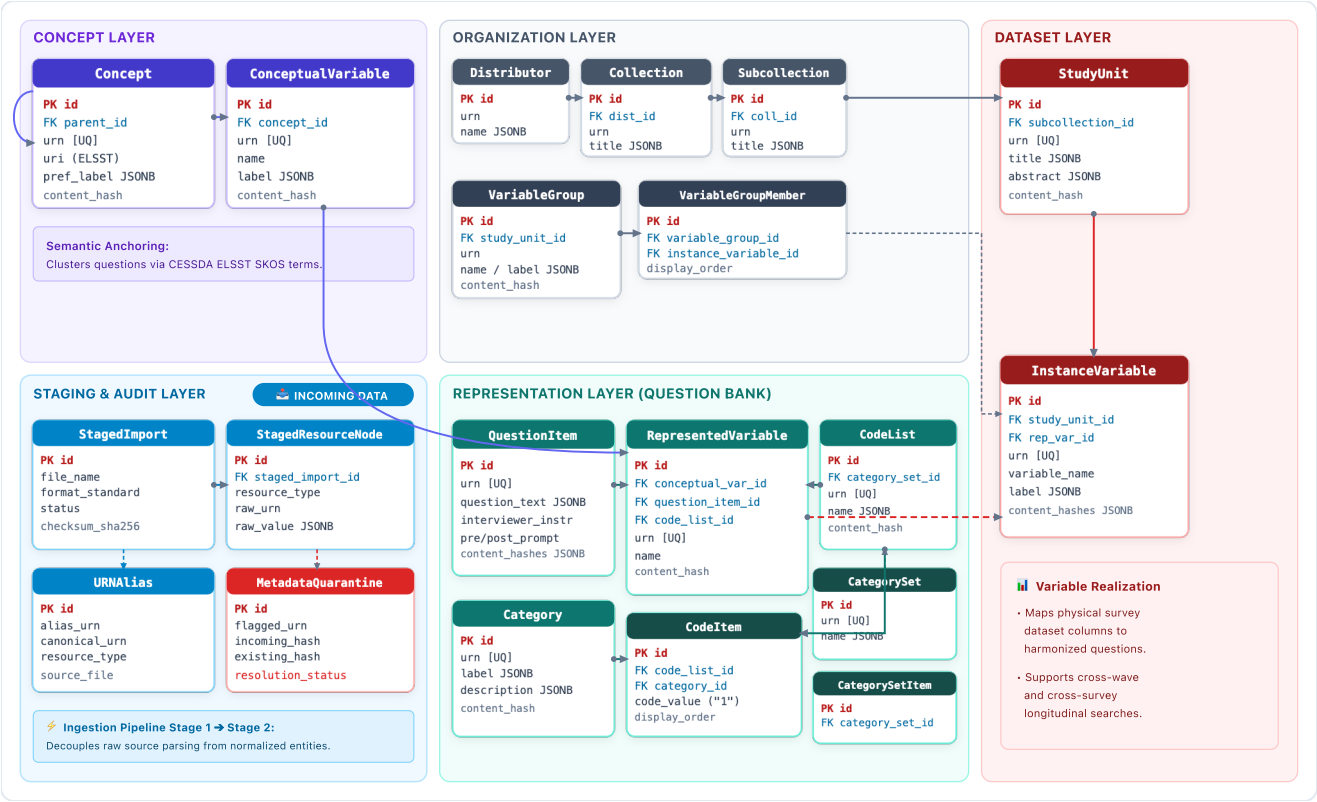
4. Upstream Tools

- **Colectica COGS:** Built Python Pydantic serializer for official DDI 4 model.
- **DDI-Toolkit:** High-performance `lxml` streamer (<50MB memory footprint).
- **[MAJOR] Parser:** Full DDI-L 3.3 → DDI 4.0 cross-standard converter & CLI.

5. Staging & Ingestion

- **Two-Stage Pipeline:** Instant HTTP 200 raw staging + async worker resolution.
- **Quarantine & Graph:** `StagedResourceNode` capture + metadata drift isolation.

2. Database Tables Entity-Relationship (ER) Model



3. DDI Streaming Ingestion & Multi-Format Staging

- **Two-Stage Decoupled Pipeline:**

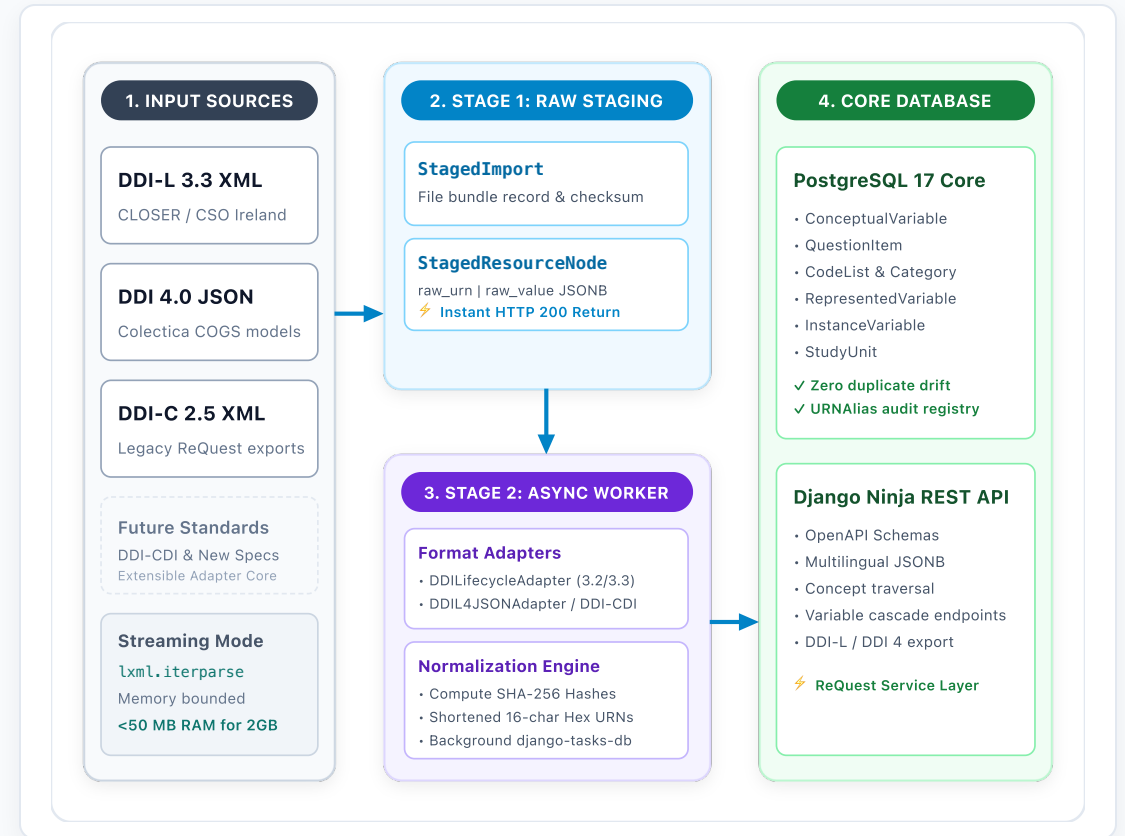
- *Stage 1 (Raw Staging):* Streaming parser ingests raw files into `StagedResourceNode` (instant HTTP 200).
- *Stage 2 (Async Worker):* Normalizes text, computes SHA-256 fingerprints, and performs entity linkage.

- **Duplicate Upload & URN Reconciliation:**

- *File Deduplication:* Re-uploaded files detected via SHA-256 to prevent redundant staging batches.
- *URN Resolution:* Differentiates canonical vs random URNs; flags duplicate URN drift to quarantine.

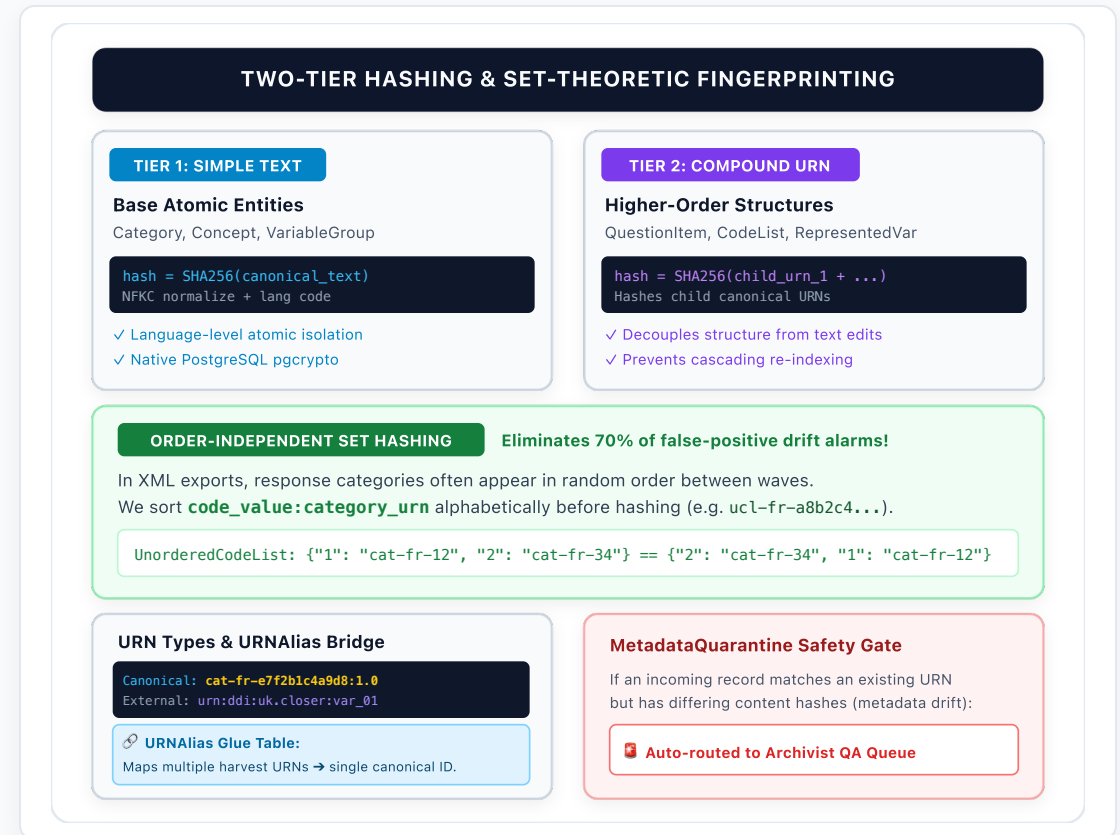
- **Multi-Standard & Profile Resilience:** Pluggable adapters support **DDI-L 3.2/3.3**, **DDI-C 2.5**, **DDI 4.0**, and remain open to **DDI-CDI**.

- **Streaming `lxml` Engine:** Bounded memory (<50MB) processing multi-gigabyte DDI XML instances.



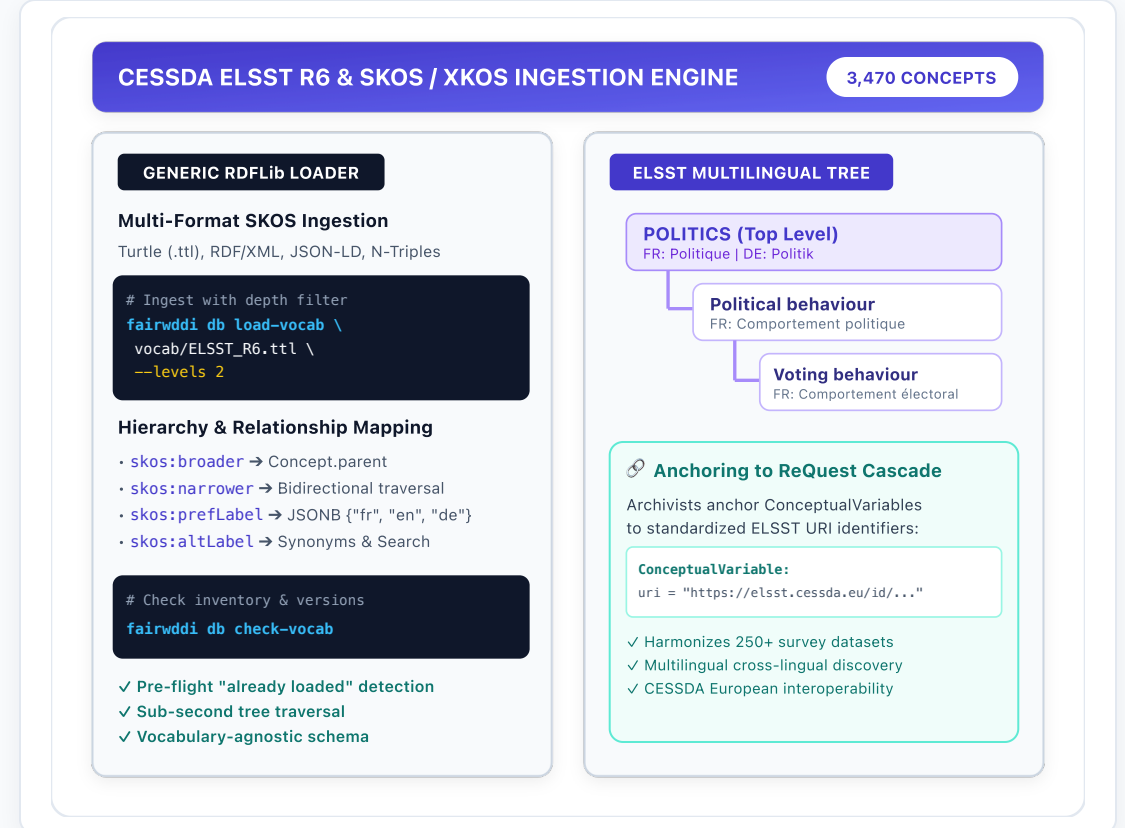
4. Content Fingerprinting & URN Strategy PHASE II FOCUS

- **Phase II Workstream:** This entire normalization, fingerprinting, and URN resolution pipeline represents the core development focus for Phase II.
- **Text Normalization:** Strips noise (spelling variants, Unicode NFKC, French punctuation, whitespace) and canonicalizes multilingual JSONB.
- **Matching by URN vs. by Content:**
 - *Matching by URN:* Direct deterministic lookup for official DDI-L agency URNs & `URNAlias`.
 - *Matching by Content:* SHA-256 fingerprint fallback for legacy DDI-C or unversioned inputs.
- **URN Taxonomy & Hashing:** Deterministic 16-char hex IDs (`cat-fr-e7f2b1c4a9d8:1.0`), two-tier hashing, and order-independent set hashing (`ucl-...`).
- **URNAlias & Quarantine:** `URNAlias` glues foreign IDs; `MetadataQuarantine` catches URN conflicts with altered content.



5. Controlled Vocabulary & SKOS/XKOS Engine

- **Automated SKOS Loader:** `fairwddi db load-vocab` parses standard RDF thesauri (`.ttl` , `.rdf` , `.jsonld` , `.nt`) into hierarchical `Concept` models.
- **CESSDA ELSST R6 Ingested:** Ingested the European Language Social Science Thesaurus (**3,470 concepts across 8 hierarchical tiers**).
- **Bidirectional Traversal:** Resolves `skos:broader` and `skos:narrower` relationships into `Concept.parent` links.
- **Audit Tooling:** Instant inventory reporting via `fairwddi db check-vocab`.
- **Multilingual Search Backbone:** Enables querying French questions using English ELSST terms.



6. Real-World Corpus Harvesting & Validation

CLOSER UK Ingestion

12 longitudinal cohort studies (**2+ GB XML, 50M+ resources**) queried via BaseX XML database.

CSO Ireland Harvest

Official statistics corpus (5 projects, 67 studies, 210 instruments, **6,044 questions**).

MIDUS Panel Data

Tested against complex longitudinal health, psychological, and well-being packages.

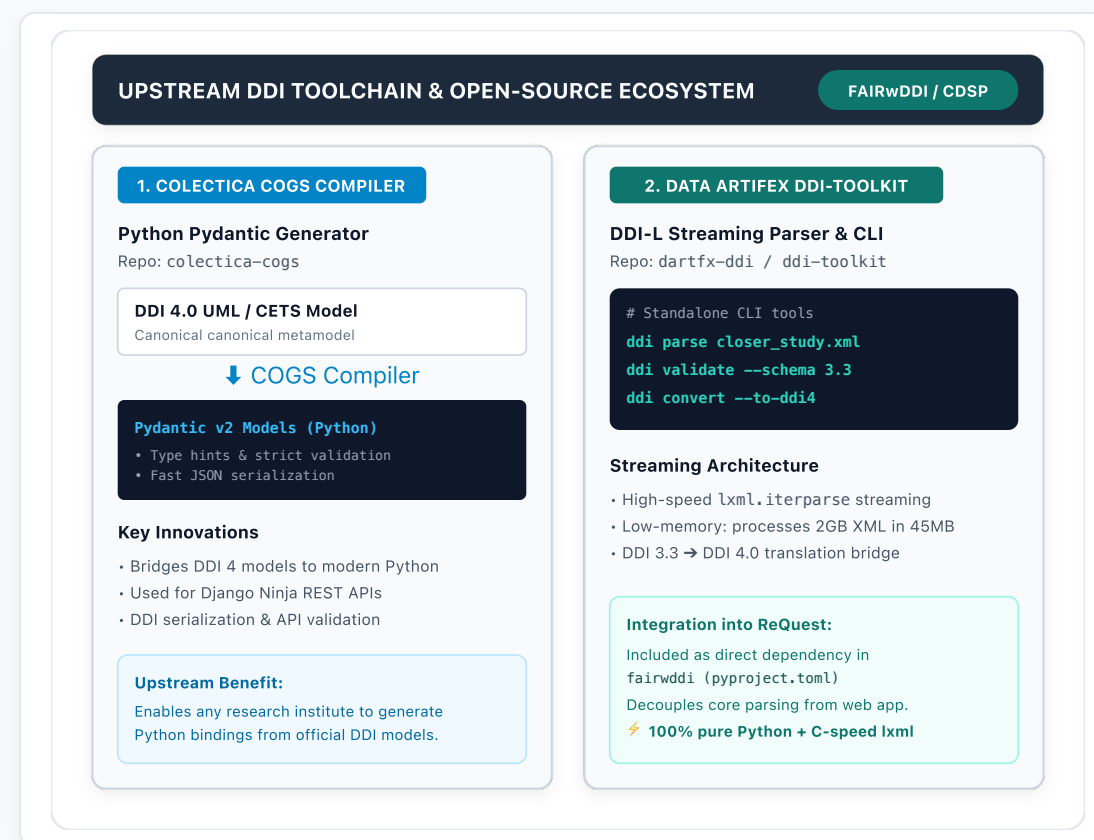
Others & EQB

European Social Survey (SIKT Norway), UKDS. Seeking European Question Bank Colectica server access.

- **Colectica Tooling Context:** Colectica is the primary toolsuite generally used across archives to manage DDI-Lifecycle metadata.
- **Corpus Storage Notice:** Harvested raw XML files are excluded from GitHub due to size (multi-GB) and licensing (available on request).
- **Steward Profiles:** Different data stewards and projects use distinct DDI profiles / resource subsets (statistics compiled across corpora).
- **Resource Relationships:** DDI-L provides multiple ways to link resources (documented 6 question ↔ variable paths in `deliverables/variable_question_relationships.md`).
- **BaseX Research DB:** Used high-speed XQuery to analyze structural schemas across millions of real-world DDI nodes.

7. Core DDI Toolchain: COGS & DDI-Toolkit

- **Colectica COGS Pydantic Generator (`colectica-cogs`)**: Extended Colectica's model compiler to auto-generate modern **Python Pydantic v2 schemas** from DDI 4 UML models.
- **Active Model & Generator Tuning**: Both the DDI 4.0 core model definitions and the COGS Pydantic code generators are still actively being refined and tweaked.
- **High-Performance DDI-L Streamer (`dartfx-ddi`)**: Dedicated streaming parser using `lxml.etree.iterparse` processing multi-GB XML in constant memory.
- **CLI & Translation Bridge**: Standalone tool for validating, inspecting, and converting DDI 3.3 → DDI 4.0 / DDI-CDI.



8. Live Demo 1: PostgreSQL Schema & Database CLI (`fairwddi db`)

- **Database Management CLI:** Complete `fairwddi db` subcommands for schema export, initialization, status, and vocabulary loading.
- **CLI User Guide:** Detailed command reference in `deliverables/cli_user_guide.md`.
- **Auto-Fallback Engine:** Automatically uses SQLite3 if PostgreSQL is not configured in `.env`.
- **Database Structure Inspection:**
 - Open live tables in **DBeaver**.
 - Review entity specifications in `deliverables/database.md`.

1. Command help & Postgres DDL

```
fairwddi db
```

```
fairwddi db export-ddl
```

2. Database initialization & wipe

```
fairwddi db init
```

```
fairwddi db wipe
```

3. Ingest ELSST & verify status

```
fairwddi db load-vocab vocab/ELSST_R6.ttl
```

```
fairwddi db status
```

9. Live Demo 2: Real-World DDI Repository & BaseX Corpus Analysis

- **Harvested DDI Corpus:** Large collection of real-world DDI 3.3 XML files in local `ddi/` directory (CLOSER UK, CSO Ireland, MIDUS, ESS SIKT, UK Data Service).
- **Repository Note:** Not committed to GitHub due to multi-GB file size and archive licensing constraints (copies available upon request).
- **BaseX XML Database Live Demo:** High-speed XQuery queries across **50M+ real-world DDI nodes**.
- **Corpus Inventory Spreadsheet:** Detailed distribution of resource types, flavours, and structural patterns across archives.

Harvested Corpora in ddi/ Folder

- CLOSER UK: 12 cohorts, 50M+ items (BaseX)
- CSO Ireland: 67 studies, 6,044 questions
- MIDUS USA: Longitudinal panel packages
- ESS SIKT & UKDS: European Social Survey

BaseX XQuery Live Inspection

for \$q in //*:QuestionItem

return count(\$q) (: 50,000+ items :)



Open: Spreadsheet with resource type counts

10. Live Demo 3: Data Artifex DDI Toolkit (`dartfx-ddi`)

- **Open-Source Toolchain:** Data Artifex DDI Toolkit (`github.com/dataartifex/ddi-toolkit`).
- **Project Structure:** Focus on the `ddilifecycle` module for high-performance streaming parsing.
- **Cross-Standard Translation Bridge:** Direct streaming conversion of DDI-Lifecycle (3.2/3.3) into DDI 4.0 JSON & XML.
- **International Community Contribution:** Tested by **SIKT (Norway)**; represents a major reusable contribution to the wider DDI community.

1. Toolkit CLI Help

```
dartfx-ddi --help
```

```
dartfx-ddi ddil324 --help
```

2. Extract DDI-L 3.3 File Statistics

```
dartfx-ddi ddil324 --stats sample_cso.ddi33.xml
```

3. Stream & Convert to DDI 4.0 (JSON/XML)

```
dartfx-ddi ddil324 --pretty sample_cso.ddi33.xml
```

11. Live Demo 4: Metadata Ingestion & Staging (`fairwddi import`)

- **Staging Pipeline:** Ingests DDI 3.3 documents into staging tables (`StagedImport` & `StagedResourceNode`).
- **Duplicate Upload & URN Handling:**
 - *Duplicate File Check:* File SHA-256 prevents redundant re-upload of identical datasets.
 - *URN Collision & Drift:* Differentiates canonical vs random URNs, binds `URNAlias` , and flags drift to quarantine.
- **Profile-Driven Filtering:** Uses YAML profiles (`profiles/request.yaml` , `all_ddi.yaml`) to specify include/exclude and other ingest options.
- **Inspection & Safeguards:** Real-time progress bar, staged node statistics, and safe relational deletion.

1. Import Help & Available Profiles

```
fairwddi import
```

```
fairwddi import list-profiles
```

2. Ingest DDI 3.3 (Checks duplicate file & URNs)

```
fairwddi import file
```

```
tests/fixtures/sample_cso.ddi33.xml
```

3. Inspect Staged Graph, URNs & Stats

```
fairwddi import list
```

```
fairwddi import stats 123
```

4. Safe Delete (Protects active relational links)

```
fairwddi import delete 123
```

12. Repository Structure & Key Deliverables Map



src/fairwddi/

- **models/ & schemas/**: 15 Django models & Pydantic v2 schemas.
- **importers/**: Stage 1 parser & profile engine.
- **cli.py**: Typer/Rich `fairwddi` CLI suite.



deliverables/ (7 Specs)

- **summary.md**: Executive synthesis.
- **database.md**: 15-table SQL schema.
- **glossary.md**: Model concepts / glossary.
- **cli_user_guide.md**: CLI user manual.
- **hashing_algorithms.md**: Fingerprinting.
- **normalization.md**: String cleaning.
- **variable_question_relationships.md**: 6 paths.



docs/ & vocab/

- **sow.md & activities.md**: SOW & logs.
- **request_overview.md**: Architecture.
- **diagrams/**: 8 SVG visualizers.
- **ELSST_R6.ttl**: SKOS thesaurus.



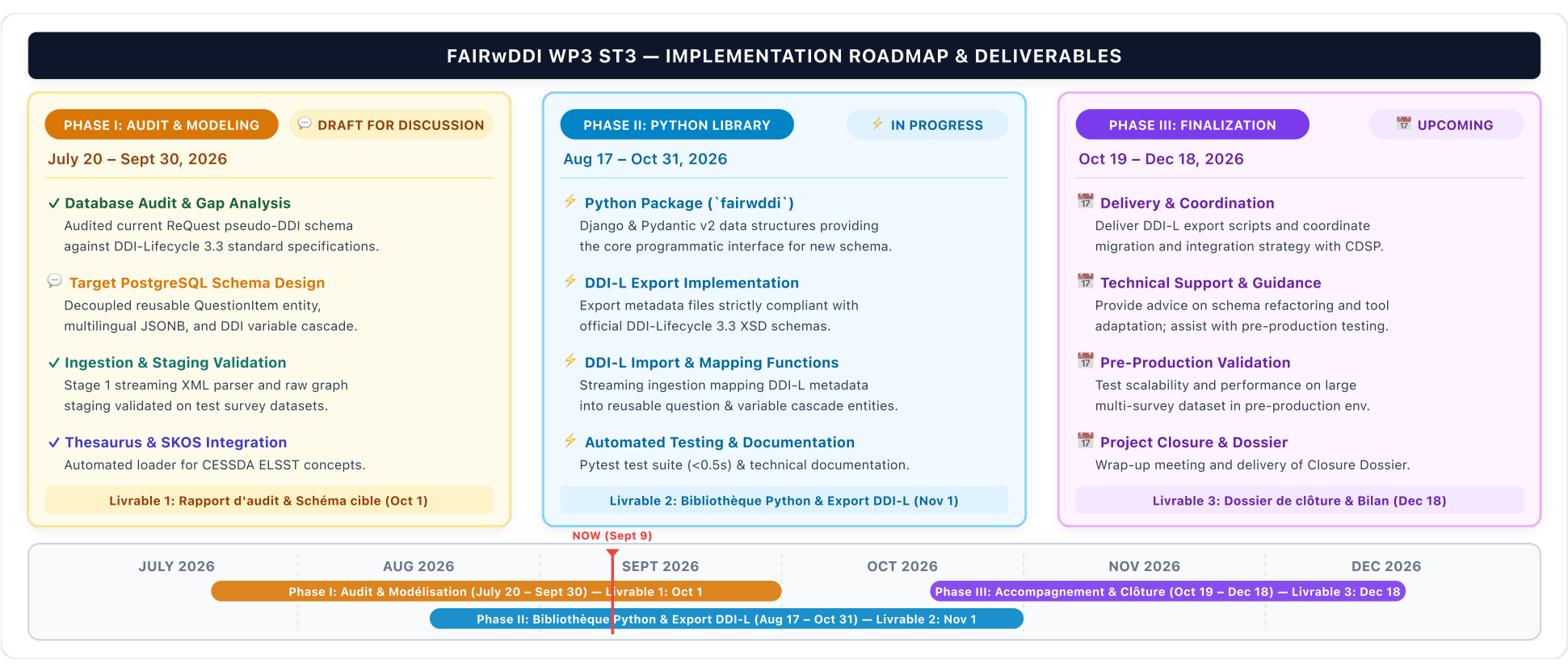
profiles/ & tests/

- **profiles/**: Staging YAML configs (`request.yaml` , `all_ddi.yaml`).
- **tests/**: Pytest test suite (<500ms) with sample DDI XML fixtures.



Ready-to-Run Environment: Managed via `uv` with `pyproject.toml` , automated type safety (`pyrefly`), standalone PostgreSQL 17 DDL export, and integrated SQLite3 local dev mode.

13. Roadmap & Upcoming Milestones (Phases II & III)



14. Next Steps, Project Needs & Discussion

Coming Up Next

- Add a command in `dartfx-ddi` to document resource relationship paths.
- Finalize model based on feedback for delivery (flexible, not set in stone).

Needs & Decisions

- Use [GitHub Discussions](#) for async feedback and questions.
- **General feedback:** Does this all make sense?
- **Time-sensitive:** Which resources do we want in the model for ReQuest (or other purposes)?
- **Phase 2:** Start thinking about normalization/harmonization strategies for Phase 2. We need one that we can implement in the context of this project; we can add more complex ones later.

DDI-Codebook Exploration

- DDI-C is out of scope but being explored and discussed.
- A DDI-C to DDI-L utility is likely the right path, but direct ingestion is also feasible.
- A harvesting tool has been added to the Data Artifex [DataVerse Toolkit](#). To be presented at October [Community Call](#).
- Has been used to collect all the datasets from `data.sciencespo.fr`.